

Understanding Large Language Model inference on Cloud Datacenter GPUs: A Quantization perspective

Afsara Benazir
University of Virginia
Charlottesville, Virginia
hys4qm@virginia.edu

Felix Xiaozhu Lin
University of Virginia
Charlottesville, Virginia
felixlin@virginia.edu

Abstract

Modern large language model (LLM) inference is increasingly driven by quantized model variants (e.g., 4-bit, 8-bit) because they promise lower memory traffic and higher throughput at acceptable accuracy. Quantized LLMs are widely adopted in cloud datacenters under the assumption that lower bit-width directly translates to higher throughput. In this work, we challenge this assumption by characterizing 26 quantization variants of LLaMA-3.3-70B on an AMD Instinct MI300X GPU using ROCm’s profiling stack. Through hardware-counter-guided analysis of large-batch prefill workloads, we show that aggressive sub-8-bit formats (e.g., IQ1_M) can incur up to 2.64x slowdown compared to Q8_0, despite reduced memory footprint. Profiling reveals that performance degradation stems not from memory bandwidth limits, but from reduced matrix-engine utilization, lower IPC (0.65 vs. 0.77), poorer memory coalescing (66.8% vs. 77.6%), and increased scalar/unpacking overhead. Overall, our results demonstrate that quantization performance is fundamentally a hardware-mapping problem: effective throughput depends more on instruction mix and compute utilization than nominal bit-width. We conclude that greater compression does not inherently guarantee better cloud inference efficiency, and deployment-aware hardware profiling is essential to understand true bottlenecks.

1 Introduction

Quantized large language models (LLMs) have become the default path to efficient inference: by storing weights (and sometimes activations) in low-bit formats (e.g., Q8, Q4, sub-4-bit), it reduces memory traffic and increase throughput while preserving acceptable accuracy [3]. Yet the notion that smaller/compressed model equals faster inference is repeatedly violated in real systems. Our prior work on Apple Silicon [1] formalized this viewpoint through a profiling-driven characterization: we compared on-device workstations, focusing on Apple Silicon and NVIDIA GPUs and used low-level runtime metrics such as ALU utilization, memory bandwidth, buffer usage, cache residency etc. to explain why aggressive compression can backfire due to dequantization overhead and reduced compute efficiency.

However, most real deployments do not happen on a single workstation. Large scale LLM inference is increasingly executed on cloud datacenter GPUs under different constraints: multi-tenant environments, datacenter CPU-GPU provisioning, driver/runtime stacks that evolve rapidly, and operating regimes (batching, throughput targets, service-level latency) that are fundamentally different from on-device local inference [4].

Broader Challenge (1) A key challenge in quantized LLM inference is that performance depends less on bit-width or peak FLOPs and more on how well a format maps to hardware pipelines and

memory systems. Ultra-low-bit schemes introduce unpacking and scalar-control overhead that can reduce vectorization, hurt coalescing, and underutilize matrix engines, even on high-bandwidth datacenter GPUs. (2) In cloud deployments, this is further complicated by large-batch prefill, throughput targets, and cost-driven utilization demands, making hardware counter-level, deployment-aware analysis essential to identify whether compute, bandwidth, or under utilization is the true bottleneck.

Motivation (1) *Cloud serving is the dominant operating point.* Prefill-heavy throughput regimes (long context, large batches) and strict cost/efficiency constraints are central in datacenter inference. On-device risks overfitting conclusions to local memory hierarchies, driver stacks, and tuning defaults that differ from what production inference uses. (2) *Generalization of quantization schemes:* We try to understand which quantization behaviors are universal (format-induced overheads), and which are platform-dependent (factors e.g. instruction set, memory hierarchy, kernel implementations etc.).

Research Questions We ask (1) Do the quantization bottlenecks we observed on single device workstations persist on cloud datacenter GPUs? (2) If so, how do their root causes change under a high bandwidth memory rich, matrix-accelerated architectures?

2 Background

LLM inference can be split into two distinct phases: **Prefill**: A large, dense, throughput-oriented phase dominated by matrix multiplications. **Decode**: A small, latency-sensitive phase with limited parallelism. While decode performance often draws attention due to its impact on user-perceived latency, prefill remains the dominant cost for large batch sizes and long contexts.

Quantization Post-training quantization (PTQ) compresses model weights to lower bit precision using either block-based schemes (e.g., GPTQ) with per-block scales/offsets and mixed precision across layers, or codebook-based vector quantization (e.g., QUIP#) that maps weight groups to learned codebook entries. At runtime, quantized values are dequantized via block scales or codebook lookups to approximate floating-point computation. Quantization changes how kernels execute (unpacking, lookup/dequant logic, instruction mix, and memory access patterns).

3 Contribution

We claim the following contributions:

- We extend our characterization of on-device LLM inference to cloud accelerators, profiling AMD MI300X on AMD Developer Cloud using ROCm stack.
- We explain quantization behaviour on cloud GPUs, linking end-to-end latency to measurable hardware effects (e.g., coalescing efficiency, IPC, instruction overhead), showing why

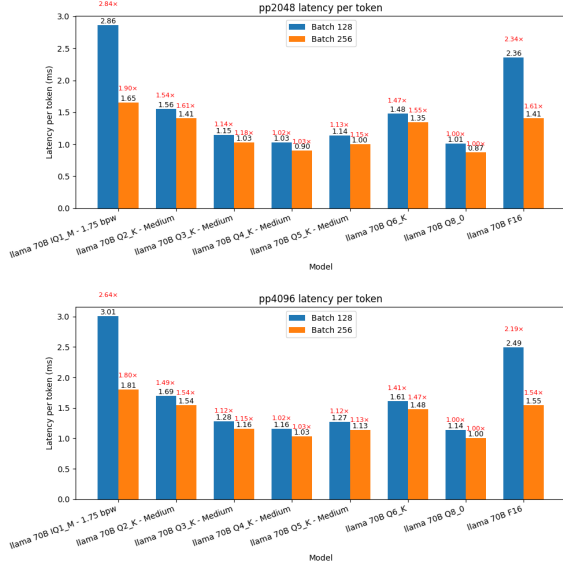


Figure 1: Per token prefill latency for LLaMA-70B across multiple quantization formats on AMD MI300X; batch sizes of 128 and 256 for prefill length of 2048 and 4096.

greater compression can reduce effective throughput even with high bandwidth memory.

- We derive a cross-tier insight: quantization is fundamentally a hardware-mapping problem; bottleneck shifts across heterogeneous platforms.

4 Methodology and Evaluation

Hardware setup Experiments run on AMD Developer Cloud with a single MI300X GPU (192 GB VRAM), 20 vCPUs, 240 GB system RAM and upto 5.3 TB/s memory bandwidth; storage includes NVMe boot and scratch disks.

Software stack We use ROCm 7.0, rocprof-compute for profiling, and llama.cpp’s HIP backend [2] to execute GGUF quantized models.

Model We primarily use LLaMA-3.3-70B (Instruct) across multiple bit precision from 1-bit to 16-bit.

Quantization Scheme We evaluate 26 quantization variants in llama.cpp [2] spanning block-based K-quants and codebook-based IQ-quants, where bits-per-weight (bpw) denotes average storage per weight. “Qx_K” indicates x-bit block quantization, while “IQx” denotes codebook-based schemes with S/M/L suffixes reflecting mixed-precision configurations that slightly vary effective bpw.

Metric We report (1) **end-to-end prefill latency** across quantization formats and prompt length/batch size to quantify the practical tradeoff between compression and throughput. (2) Several **low level hardware metrics**: speed-of-light utilization (achieved vs. peak compute and memory engines), wavefront runtime (active vs. stall cycles), instruction mix (VALU, SALU, MFMA, VMEM, branch), arithmetic operations (floating-point vs. integer ops), IPC (instructions per cycle), effective memory bandwidth, cache utilization (L1 and L2), vL1D coalescing efficiency and roofline metrics (arithmetic intensity vs. achieved throughput).

Metric (ROCm Counter)	LLaMA-70B IQ1_M	LLaMA-70B Q4_K_M
vL1D Coalescing (%)	66.84%	77.62%
vL1D BW Utilization (% peak)	10.15%	7.91%
vL1D Cache BW (GB/s)	8290.67	6465.68
IPC	0.65	0.77

Figure 2: Impact of quantization format on memory access efficiency and execution throughput

Results The x-axis in Figure 1 shows quantization precision (1–16 bits) and the y-axis reports per-token latency, with red labels indicating slowdown relative to the Q8_0 baseline (1.00x at batch size 128). While Q8_0 delivers the lowest latency, IQ1_M reaches 3.01 ms per token, incurring a 2.64x slowdown. The observed trends are counterintuitive — and they reveal a great deal about how prefill behaves on MI300X.

Sub-8-bit formats significantly reduce memory footprint but also introduce complex bit unpacking, extra integer/scalar work, and reduced use of wide vector paths, lowering useful compute per issued instruction in GEMM-heavy prefill.

ROCm counters attribute this slowdown to reduced memory efficiency and higher overhead in Figure 2: vL1D coalescing drops from 77.62% (Q4_K_M) to 66.84% (IQ1_M), despite high reported cache bandwidth (e.g., 8290.67 GB/s), & IPC decreases (0.77 to 0.65).

We analyze floating-point (FP) and integer operations to determine whether prefill is compute or memory bound: IQ1_M executes substantial FP work but incurs higher scalar overhead, resulting in low active utilization (~1–5% of peak), whereas Q4_K_M issues ~4x more MFMA instructions and better engages matrix engines. Despite large prefill (70Bx2048), memory remains underutilized (L1 ~15%, L2 ~7%, fabric <1 TB/s), confirming the workload is not bandwidth-bound but limited by instruction mix and matrix-engine utilization.

5 Plan

As observed, across cloud prefill experiments, more aggressive quantization does not guarantee higher throughput. To further analyze whether performance bottlenecks stem from memory bandwidth, compute limits, synchronization, or pipeline stalls, we will analyze (1) hardware utilization ceilings (speed-of-light views) to assess overall compute efficiency; (2) pipeline behavior, using wavefront activity and stall breakdowns to determine scheduling and dependency overheads and (3) we will add explicit instruction-level attribution (e.g., VALU/SALU/MFMA/VMEM/branch mix and MFMA issue volume). This analysis will help determine where utilization is lost rather than whether peak hardware limits are reached.

References

- [1] Afsara Benazir and Felix Xiaozhu Lin. Benchmarking and characterization of large language model inference on apple silicon. *Proceedings of the ACM on Measurement and Analysis of Computing Systems*, 9(3):1–26, 2025.
- [2] Georgi Gerganov. <https://github.com/ggml-org/llama.cpp>, 2023. URL <https://github.com/ggml-org/llama.cpp>. Accessed: 2026-03-02.
- [3] Renren Jin, Jiangcun Du, Wuwei Huang, Wei Liu, Jian Luan, Bin Wang, and Deyi Xiong. A comprehensive evaluation of quantization strategies for large language models. In *Findings of the association for computational linguistics: ACL 2024*, pages 12186–12215, 2024.
- [4] Xiaoyu Ma and David Patterson. Challenges and research directions for large language model inference hardware. *arXiv preprint arXiv:2601.05047*, 2026.